

Scenario Controller — Implementation Report

Project: STING Bioinformatics (VR Nursing / Informatics Training)

Engine: Unity 6000.4.9f1 · URP · OpenXR · Meta XR · Interaction via BNG VRIF

Assembly: `Assembly-CSharp` · global namespace · `Assets/Scripts/Scenario/`

Date: 2026-07-27

Status: Implemented and compile-verified. Not yet runtime-tested.

1. Executive Summary

The Scenario Controller specified in *The Scenario Controller — Architecture & Design Report* (2026-07-22) has been implemented in full: all 14 files from the design report's §10 file manifest, plus the one backward-compatible edit to `Quiz.cs` that the manifest calls for.

The system is a **data-oriented linear sequencer**. A scenario is an ordered list of step assets. The controller walks the list one step at a time; each step signals when it is done; the controller advances. It fires one completion event at the end. Designers assemble and reorder scenarios entirely in the Inspector. Engineers add capabilities by writing one small class, never by editing the controller.

Three things are worth stating up front, because they shape every recommendation in §8:

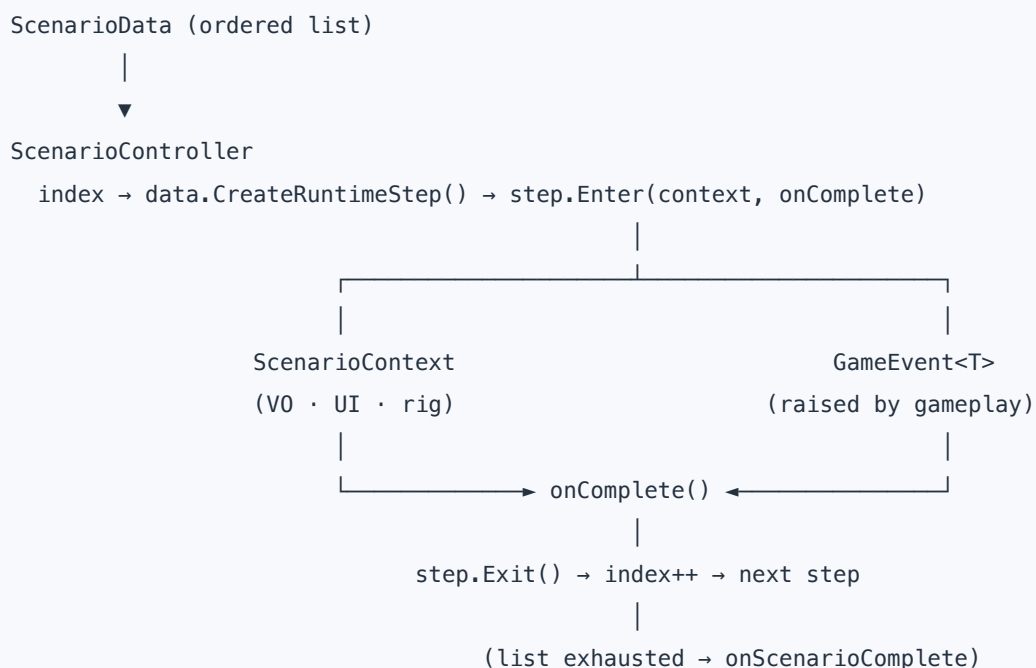
1. **The code compiles clean against the project's real reference assemblies** — zero errors, zero substantive warnings. It has **never been run**. Compile-clean is not the same as working.
 2. **Two behaviours that exist today have no home in the new system**: end-of-run results aggregation, and per-answer feedback *text*. Switching Hospital Room over without addressing these is a silent regression.
 3. **Two live defects were found during implementation** — one pre-existing in `Quiz.cs` that will make the new system look broken when it isn't, and one in new code on the restart path. Both are cheap to fix and are detailed in §6.3.
-

2. What the System Is

2.1 The seven pieces

Piece	Kind	Responsibility
ScenarioData	ScriptableObject asset	The designer-editable scenario: an ordered <code>List<ScenarioStepData></code>
ScenarioStepData	abstract ScriptableObject	Base for every step asset; holds only that step's data
IScenarioStep	runtime interface	The step's behaviour: <code>Enter(ctx, onComplete) / Exit()</code>
ScenarioController	MonoBehaviour	Runs the list: enter → wait for completion → exit → advance
ScenarioContext	serialized class	Shared references and services handed to every step
GameEvent / GameEvent<T>	ScriptableObject channel	Observer signalling between gameplay and steps
Panel prefabs	Prefabs	Self-contained UI shown per step

2.2 The control loop



The controller holds an index. It asks the current `ScenarioStepData` to build its runtime executor, calls `Enter(context, onComplete)`, and does nothing else. When the step invokes `onComplete`, the controller calls `Exit()`, increments the index, and enters the next — or fires `onScenarioComplete` if the list is exhausted.

2.3 Evidence the controller is genuinely decoupled

This is the central architectural claim, so it is worth measuring rather than asserting. A scan of `ScenarioController.cs` for referenced types yields exactly four:

```
IScenarioStep · ScenarioContext · ScenarioData · ScenarioStepData
```

No `AudioSource`, no `Quiz`, no `GameObject`, no `Grabbable`, no `Canvas`. The controller cannot break when audio, UI, or interaction changes, because it cannot see them. Likewise, a scan of all 14 files for `FindObjectOfType`, `FindAnyObjectByType`, and `.Instance` returns zero hits outside a comment. There are no hidden globals.

3. What Was Implemented

3.1 New files — 14, 866 lines

Assets/Scripts/Scenario/Core/

File	Lines	Contents
<code>IScenarioStep.cs</code>	14	The step contract. <code>Enter</code> once, <code>onComplete</code> exactly once, then <code>Exit</code>
<code>ScenarioStepData.cs</code>	15	Abstract SO base; <code>designerNote</code> field; abstract <code>CreateRuntimeStep()</code>
<code>ScenarioData.cs</code>	21	<code>List<ScenarioStepData></code> ; <code>StepCount</code> ; bounds-safe <code>GetStep(index)</code>
<code>ScenarioContext.cs</code>	95	Shared services + the single VO audio path (<code>PlayVoice</code> / <code>StopVoice</code>)
<code>ScenarioController.cs</code>	156	The linear runner, completion latches, <code>Begin</code> / <code>Restart</code> / <code>StopScenario</code>

Assets/Scripts/Scenario/Events/

File	Lines	Contents
<code>GameEvent.cs</code>	43	Parameterless observer channel as an asset
<code>GameEventGeneric.cs</code>	37	<code>GameEvent<T></code> abstract base
<code>StringGameEvent.cs</code>	18	Typed channel for task/item ids; <code>RaiseString(string)</code> for UnityEvents
<code>SceneEventRelay.cs</code>	50	Adapter: channel → scene-bound UnityEvent; <code>ReportComplete()</code>
<code>TaskEventRaiser.cs</code>	52	Adapter: gameplay UnityEvent → channel

Assets/Scripts/Scenario/Steps/

File	Lines	Contents
<code>NarratorStepData.cs</code>	42	VO clip + transcript; completes on clip end
<code>WaitForTaskStepData.cs</code>	87	Completes when a matching task id is raised; optional prompt VO
<code>InvokeSceneEventStepData.cs</code>	88	Raise a channel; optionally wait for external completion
<code>UIQuestionStepData.cs</code>	148	Question + validation + retry policy + feedback VO

3.2 Modified files — 2

`Assets/Scripts/Quiz.cs` — the one edit the design report's §10 calls for.

Added `public event Action<int> AnswerSelected`. Button clicks now route through a new `RaiseAnswerSelected(int)` method that fires the event **and** still calls the legacy `scenario.OnAnswerSelected(index)` — now null-guarded, which it was not before. `SampleSceneV6` continues to work unchanged; Hospital Room no longer throws a `NullReferenceException` for want of a `SceneManager` in the scene.

`Assets/Scripts/Main Menu/MainMenuManager.cs` — absorbed the menu half of `GameManager`.

Gained `ShowMainMenu()`, `ShowSettings()`, `HideMenu()`, and panel placement. Position and rotation are now serialized fields instead of hardcoded constants. All panel references are null-guarded so the component remains valid in the Main Menu scene, where no panels are assigned. `LoadScene()` and `ExitApplication()` are unchanged.

3.3 Removed files — 1

`Assets/Scripts/GameManager.cs` (and its `.meta`) — deleted.

`GameManager` was doing two unrelated jobs: menu panel toggling, and `BeginTraining()`, which was the scenario entry point (its body was the comment `// To do: Officially start training`). These were split rather than merged into the controller, because the design report's §4 is explicit that the controller never references UI types.

- `BeginTraining()` → `ScenarioController.Begin()`
- `ResetExperience()` → `ScenarioController.Restart()`
- Panel toggling → `MainMenuManager`, which already sat on the same prefab and already owned menu concerns

Consequence: four UnityEvent button bindings now point at a missing component and must be re-bound in the editor. See §9.1.

3.4 Verification performed

The project's own `Assembly-CSharp.csproj` reference list (419 assemblies) was extracted and used to compile the new code with Roslyn against Unity 6000.4.9f1's actual reference assemblies, at `langversion:9.0`.

Scope	Result
Focused build — <code>Scenario/*</code> + <code>Quiz</code> + <code>QuestionSO</code> + <code>ScenarioManager</code> + <code>ResultUi</code> + <code>MainMenuManager</code>	0 errors. Output DLL produced
Full <code>Assembly-CSharp</code> (245 source files)	0 errors in new or modified code
Warnings in new code	Only <code>CS0649</code> — <code>[SerializeField]</code> fields Roslyn cannot see the Inspector assigning. Unity suppresses these
Name collisions	None. Generic type names (<code>GameEvent</code> , <code>ScenarioData</code> , <code>TaskEventRaiser</code>) do not clash with BNG, Meta XR, or any package in <code>Assembly-CSharp</code>

The full build reports 7 remaining errors, **all pre-existing and unrelated** — `UnityEngine.XR.InputDevice` and `TrackingOriginModeFlags` resolution inside `BNG Framework/Scripts/Core/InputBridge.cs` . These stem from how the stale `.csproj` records BNG's assembly references and do not reflect a real problem in Unity's own compilation.

3.5 Verification *not* performed — read this

The system has never been executed. There has been no play-mode run, no scene wiring, and no device test. Specifically unverified:

- Whether VO timing feels correct against real narration clips
- Whether `GameEvent` listener lifetimes behave correctly across Unity's *Enter Play Mode Options* when domain reload is disabled
- Whether the `UIQuestionStep` → `Quiz` panel handoff renders correctly in world space on the EHR terminal
- Any XR-specific interaction timing

Treat §4 through §7 as design analysis, not as test results.

4. Isolation — How This Avoids Interfering With Existing Systems

This was an explicit requirement, so each existing system is addressed individually.

System	Files touched	Coupling introduced
BNG VRIF (Grabbable, InputBridge, HandPoser)	None	One-way and optional. A <code>Grabbable</code> 's <code>UnityEvent</code> calls <code>TaskEventRaiser.Raise()</code> , which raises a channel. BNG has no knowledge of the scenario. No BNG file was modified
EHR state machine (<code>EHRContext</code> + 6 states)	None	Zero. Entirely independent. If the scenario should later drive EHR states, that goes through <code>InvokeSceneEventStep</code> → <code>SceneEventRelay</code> → <code>EHRContext.ChangeState</code> , with no code change to either side
Settings / audio (<code>SettingsManager</code> , <code>MainAudioMixer</code>)	None	Zero code dependency. The only link is an Inspector field: the scenario's <code>AudioSource</code> outputs to the <code>Voices</code> mixer group. Volume control remains entirely with <code>SettingsManager</code>
XR / OpenXR / Meta XR	None	Zero
Interaction, Drag, Pickup scripts	None	Zero
Quiz / ScenarioManager	<code>Quiz.cs</code> (additive)	The one real overlap. Handled by an additive event plus a null guard — see §4.1
Main menu / settings UI	<code>MainMenuManager.cs</code>	The one breaking change. Four button rebinds — see §9.1

4.1 The one overlap, and the one hard rule

`ScenarioManager` and `ScenarioController` are both sequencers, and both can respond to the same `Quiz` answer. The compatibility shim in `Quiz.RaiseAnswerSelected` deliberately fires *both* the new event and the legacy direct call, so `SampleSceneV6` keeps working untouched during migration.

Hard rule: never place a `ScenarioManager` and a `ScenarioController` in the same scene. Both would respond to a single click. The shim is a migration bridge, not a coexistence strategy. When Hospital Room is proven, delete `ScenarioManager.cs` and the shim together.

Hospital Room has never contained a `ScenarioManager` — only `SampleSceneV6` does — so the migration cost is genuinely low.

4.2 Safe defaults

`beginOnStart` defaults to **false**. A `ScenarioController` dropped into a scene does nothing until something explicitly calls `Begin()`. It cannot interfere by simply existing.

5. Strengths

Additive, and cheap to reverse. Fourteen new files in their own folder; two existing files modified; one deleted. Full revert is `rm -rf Assets/Scripts/Scenario` plus three git checkouts. Nothing is entangled.

The controller genuinely cannot see the rest of the game. Four referenced types, all scenario-internal (§2.3). This is the Open/Closed payoff made concrete: adding a "wait N seconds" step means one new data class and one new runtime class, and the controller is not recompiled in spirit or in fact.

Single-shot completion is enforced twice, independently. Each step's callback carries a closure-captured latch *and* a `ReferenceEquals(currentStep, step)` identity check. The first stops a step that calls `onComplete` twice in a row; the second stops a *late* callback arriving from a step the controller already exited. Either guard alone would leave a gap.

No polling anywhere. Zero `Update()` methods across all 14 files. Steps sleep until an event or callback wakes them. On a Quest this matters.

Fail-open on missing data. A missing clip, an unassigned channel, or a null step logs an error and *completes* rather than hanging. For a team wiring ScriptableObjects in the Inspector, a silently stalled scenario is far harder to diagnose than a red line in the console.

No hidden globals. Every dependency arrives through `ScenarioContext`. A step's needs are visible in one Inspector block.

Designer-authorable end to end. Reorder by dragging. Reuse a narration clip, a question, a channel, or a panel prefab across many scenarios. Reordering cannot break compilation.

6. Weaknesses, Risks, and Defects

6.1 Accepted design limits

These are deliberate, inherited from the design report, and are listed so nobody rediscovers them as surprises.

No branching. Linear by design (§9 of the design report). The major-selection flow *does* require branching, so a layer above the controller is needed — see §8.3.

Synchronous completion recurses. A step that completes inside `Enter()` (a fire-and-forget scene event) advances the sequence through the call stack. Bounded by step count. At a realistic 20–40 steps this is a non-issue; it would only matter for thousands of instantly-completing steps.

One context per controller. `ScenarioContext` is serialized inline on the controller, so it cannot be shared between two controllers. Correct for one linear scenario; a limit if that ever changes.

No assembly definition. Everything lives in `Assembly-CSharp` in the global namespace, as the design report specifies. This means no compile isolation and slower iteration, and it puts generic names like `GameEvent` and `ScenarioData` in a namespace shared with every unnamespaced asset in the project. No collision exists today (verified in §3.4), but importing an asset pack that defines its own `GameEvent` would break the build. A namespace or an `.asmdef` is cheap insurance later.

6.2 Implementation risks

VO waits use scaled time. `ScenarioContext.PlayVoice` waits `clip.length` via `WaitForSeconds`, which is affected by `Time.timeScale`. If a pause menu is ever added that sets `timeScale = 0`, **every narration step will hang forever**. A project-wide search confirms `timeScale` is not currently used anywhere, so this is latent rather than active — but a pause feature is a natural thing to add to a VR training module. Mitigation is one word: `WaitForSecondsRealtime`.

VO completion is estimated, not observed. Waiting on `clip.length` rather than polling `AudioSource.isPlaying` means the callback can drift from reality if the source is paused externally, or if the clip is a streaming or heavily compressed asset. Low risk with short pre-baked narration clips; worth revisiting if VO gets long.

ScriptableObject state across play-mode reloads. `GameEvent.OnDisable()` clears its listener list, which is the documented safeguard. This has not been tested with *Enter Play Mode Options* and domain reload disabled, a configuration that changes ScriptableObject lifetimes. Worth an explicit test before relying on it.

6.3 Defects found — both should be fixed

D-1 · `Quiz.ShowQuestion` stops early on short questions. Pre-existing. Live today.

`Assets/Scripts/Quiz.cs` , inside the button-setup loop:

```
bool active = i < question.GetAnswerCount();
answerButtons[i].SetActive(active);
if (!active) return;      // ← should be: continue
```

The `return` exits the whole loop at the first inactive button, so every button *after* it keeps whatever state it had from the previous question — still visible, still showing stale text, still wired to a stale answer index.

This is not hypothetical. Answer counts across the question assets are uneven:

Asset	Answers
<code>Question 1 – Question 6</code> , <code>PostQ_Nurse</code> , <code>PostQ_Inftics</code>	4
<code>MethoAlert_Q</code>	3
<code>SelectMajor_Q</code>	2

Showing `SelectMajor_Q` (2 answers) after any 4-answer question leaves button index 3 on screen, bound to `AnswerSelected(3)` — a phantom option that reports an answer index the question does not have. `MethoAlert_Q` (3 answers) leaves one stale button in the same way.

Because this lands squarely on the major-selection question, it will look like the new scenario system is broken when the fault is a one-word bug that predates it. **Recommended fix: change `return` to `continue`.**

D-2 · `TaskEventRaiser.onlyOnce` is not reset by `Restart()` . New code.

`TaskEventRaiser` defaults to `onlyOnce = true` so a repeatedly grabbed item does not spam its channel. That flag persists for the object's lifetime. On `ScenarioController.Restart()` , an item the player already interacted with will refuse to raise again — and the `WaitForTaskStep` waiting on it **waits forever**. The scenario dead-ends with no error.

The class exposes `ResetRaised()` , but nothing calls it. Recommended fix, in order of preference:

1. A `GameEvent` "scenario reset" channel that every `TaskEventRaiser` subscribes to, raised by `Begin()` . Stays in-pattern, requires no new coupling.
2. Failing that, have whatever drives restart call `ResetRaised()` on the relevant raisers.

This only affects the restart path, so it will not appear in a first straight-through test — which is exactly why it is worth fixing before anyone demos a second run.

7. Gaps Against Current Behaviour

`ScenarioManager` (214 lines) does three things. The new system replaces one of them cleanly and does not yet cover the other two. The design report's file manifest does not mention either gap.

Responsibility	Covered by new system?
Question sequencing	Yes — <code>ScenarioData</code> + <code>UIQuestionStep</code> , and more flexibly
Branching on chosen major	No — deliberate. Needs a layer above the controller (§8.3)
End-of-run results aggregation	No gap coverage at all
Per-answer feedback <i>text</i>	No — the new step plays feedback VO only

Results aggregation. `ScenarioManager` accumulates `answerCorrectness` and `wrongFeedback` across the whole run, then produces either *"Safe to administer. All selections matched safe clinical practice."* or a bulleted patient-safety-risk summary. This is the pedagogical payoff of the entire module. `UIQuestionStep` validates an answer and forgets it — nothing accumulates.

Feedback text. `ScenarioManager` shows `FeedbackPanel` + `FeedbackText` for a fixed 10 seconds after each answer. `UIQuestionStep` plays feedback *audio* only. The `FeedbackPanel` inside `QuestionPrefab` would go unused.

Switching Hospital Room to `ScenarioController` today loses both, silently. **These are the blocking items**, not branching.

8. Recommendations and Next Steps

Ordered. Each phase is independently testable, which is the point — the guiding constraint here is *minimise mistakes*, and the way to do that is to never introduce two unknowns at once.

Phase 0 — Fix the two defects first (small, high value)

1. `Quiz.cs`: `return` → `continue` (D-1). One word. Prevents a phantom answer button from making everything downstream look broken.
2. Wire a scenario-reset channel for `TaskEventRaiser` (D-2). Prevents a dead-end on the second run.
3. Optionally, `WaitForSeconds` → `WaitForSecondsRealtime` in `ScenarioContext`. Pre-empts a pause menu breaking all narration later.

Phase 1 — Close the behavioural gaps

`ScenarioResults` as a `ScriptableObject` record. `Record(bool correct, string feedback)`, `Clear()`, and summary getters. `UIQuestionStep` writes to it through an optional field, so steps without one still work. A new

`ShowResultsStepData` reads it and drives the existing `ResultsUI`.

Why an asset rather than a manager: steps write to a shared record instead of reaching for a singleton, which preserves the no-hidden-globals property; the record survives a branch, since both major tracks write to the same asset; and "show results" becomes just another authorable step, so designers choose where it appears.

One footgun to handle explicitly: `ScriptableObject` state persists across play sessions in the editor, so `Begin()` must call `Clear()`.

Feedback text on `Quiz`, not on the context. Add `ShowFeedback(string)` / `HideFeedback()` to `Quiz`. The `FeedbackPanel` is already a child of `QuestionPrefab`, which `Quiz` owns, so `ScenarioContext` does not need to grow. Dwell on the feedback clip's length, falling back to a serialized seconds value when there is no clip — strictly better than the current fixed 10 seconds.

Phase 2 — Prove the loop before adding anything

Wire Hospital Room with a **single flat** `ScenarioData`, no branching:

```
Narrator (intro) → UIQuestion ×N → WaitForTask (scanner) → InvokeSceneEvent → ShowResults
```

Run it end to end. This is where the unverified items in §3.5 get verified. Do not add branching until this runs clean.

Phase 3 — Branching, as a layer above the controller

The design report's §9 already ruled on this: branching *"belongs in a separate layer, not bolted onto this controller."*

The branch here is not a general graph. It is a single two-way split at a known point, into two terminal tracks. `SelectMajor_Q` already carries the branch key in its answer text — `"Nursing"` and `"Healthcare Informatics / Computing"`, with `correctAnswerIndex: All` so every answer passes.

Proposed shape:

- Three `ScenarioData` assets: `Common` (intro → general questions → `SelectMajor_Q`), `Track_Nursing`, `Track_Informatics`
- `UIQuestionStepData` gains an optional `StringGameEvent answerChannel` that raises `question.GetAnswer(selectedIndex)`. **`SelectMajor_Q` needs no changes at all**
- A small `ScenarioDirector` `MonoBehaviour` listens on that channel, remembers the pick, and on `onScenarioComplete` calls `controller.Begin(track)`
- `ScenarioController` gains a `Begin(ScenarioData)` overload — purely additive; the controller stays linear and ignorant

Roughly 70 lines. The common prefix is authored once.

The alternative, and why not. Two complete scenario assets with the major chosen in the main menu needs zero new code — but it duplicates the common prefix across two assets. On a repository with 37 branches and a rotating team, someone will edit the intro in one and not the other. It also moves major selection out of the experience and into the menu, which is a UX change nobody asked for. Only worth it if the major should genuinely be picked before training starts.

Phase 4 — Retire the old path

Once Hospital Room is proven: delete `SceneManager.cs`, and delete the compatibility shim in `Quiz.RaiseAnswerSelected` at the same time. They are a matched pair. Leaving the shim after `SceneManager` is gone is harmless but misleading; leaving `SceneManager` while a `ScenarioController` exists in the same scene is the failure mode §4.1 warns about.

8.1 On keeping it simple

The stated requirement is a system that is simple, does the job, and stays out of the way. Three things protect that, and they are worth defending against future pressure:

1. **Do not add features to `ScenarioController`**. Its value is that it references four types. Every capability belongs in a step. If a change requires editing the controller, that is a signal the change belongs somewhere else.
2. **Do not let steps talk to each other**. They share state only through `ScenarioContext` or a channel asset. Step-to-step references would rebuild the coupling this design exists to remove.
3. **Resist a branching graph**. A single director handling one two-way split is ~70 lines. A general graph system with a custom editor is a different project, and the design report deliberately excluded it.

9. Editor Wiring Checklist

9.1 Rebind the four orphaned buttons

`GameManager` no longer exists, so these UnityEvent bindings show as missing:

Prefab	Line	Was	Now
UI/Main Menu Panel.prefab	1180	<code>BeginTraining</code>	<code>ScenarioController.Begin()</code> + <code>MainMenuManager.HideMenu()</code>
UI/Main Menu Panel.prefab	636	<code>MainMenuToSettings</code>	<code>MainMenuManager.ShowSettings()</code>
UI/Settings Panel.prefab	3520	<code>SettingsToMainMenu</code>	<code>MainMenuManager.ShowMainMenu()</code>
UI/Settings Panel.prefab	5904	<code>ResetExperience</code>	<code>ScenarioController.Restart()</code>

Also re-assign `mainMenuPanel` and `settingsPanel` on the `MainMenuManager` component of the `GameManager` prefab, and remove the now-missing `GameManager` component from that prefab.

9.2 Two things the scene does not have yet

No `AudioSource` anywhere. Both `Hospital Room.unity` and `SampleSceneV6.unity` contain zero. `ScenarioContext.voiceSource` needs one, with its **Output** set to the **Voices** group on `MainAudioMixer` — that is the group carrying the exposed `NarrationVolume` parameter that the settings sliders drive.

The scanner has no `Grabbable`. Hospital Room contains exactly one `Grabbable` in the entire scene. `TaskEventRaiser.Raise()` needs a grab event to hang off before `WaitForTaskStep` can ever complete.

9.3 Asset creation menus

Menu path	Creates
Scenario/Scenario	ScenarioData
Scenario/Steps/Narrator	NarratorStepData
Scenario/Steps/UI Question	UIQuestionStepData
Scenario/Steps/Wait For Task	WaitForTaskStepData
Scenario/Steps/Invoke Scene Event	InvokeSceneEventStepData
Scenario/Events/Game Event	GameEvent
Scenario/Events/String Game Event	StringGameEvent

9.4 Minimum viable wiring

1. Create a `ScenarioController` GameObject in Hospital Room
 2. Create one `ScenarioData` asset; add step assets in order
 3. Add an `AudioSource` → `Voices` group; assign to `context.voiceSource`
 4. Assign the quiz panel root to `context.pcUiRoot` and the `Quiz` component to `context.quiz`
 5. Leave `beginOnStart` off; call `Begin()` from the Begin Training button
 6. For each waited-on task: add `Grabbable` + `TaskEventRaiser` to the prop, set the task id, point it at a `StringGameEvent`, and give the matching `WaitForTaskStepData` the same channel and id
-

10. Appendix — Design Patterns in Use

Pattern	Where	What it buys
Strategy	<code>IScenarioStep</code> + concrete steps	The controller runs against one interface
Factory Method	<code>ScenarioStepData.CreateRuntimeStep()</code>	The data asset picks its runtime class; no type switch
Type Object	<code>ScenarioStepData</code> vs <code>IScenarioStep</code>	Designers create step <i>types</i> as assets
Observer	<code>GameEvent</code> / <code>GameEvent<T></code>	Gameplay raises without knowing listeners
State (linear)	<code>Enter()</code> / <code>Exit()</code>	Clean entry/exit; mirrors the project's EHR state pattern
Command	<code>InvokeSceneEventStep</code>	Encapsulates "invoke this" without knowing the receiver
Context Object	<code>ScenarioContext</code>	One hand-off of shared services
Dependency Injection	<code>Enter(ctx, onComplete)</code> , <code>context.Runner = this</code>	Steps receive dependencies and their continuation
Continuation-passing	<code>Action onComplete</code>	Async completion without <code>Update()</code> polling
Adapter	<code>TaskEventRaiser</code> , <code>SceneEventRelay</code>	Bridges UnityEvent world ↔ channel world
ScriptableObject Architecture	data, steps, channels as assets	Configuration and wiring live as project assets

11. Summary of Current State

Implemented	14 files, 866 lines, matching the design report's §10 manifest exactly
Modified	<code>Quiz.cs</code> (additive event + null guard), <code>MainMenuManager.cs</code> (absorbed menu duties)
Removed	<code>GameManager.cs</code>
Compiles	Yes — 0 errors, 0 substantive warnings, against Unity 6000.4.9f1 reference assemblies
Runs	Not yet tested
Blocking gaps	Results aggregation; per-answer feedback text
Known defects	D-1 <code>Quiz.ShowQuestion</code> early return (pre-existing, live); D-2 <code>TaskEventRaiser</code> restart dead-end
Interference with other systems	None. BNG, EHR, XR, settings, and audio are untouched
Immediate next step	Phase 0 defect fixes, then Phase 1 results record